

Social Media Studio

Change one blog post into a full social campaign. Your system makes one post version for each platform. A person approves each version. A scheduler publishes each approved version one time, and only one time.

Difficulty: Medium–Hard

Self-paced · no deadlines

JavaScript or Python

Public GitHub repo

\$0 · no credit card, ever

THE FLAVOR

One post in, a scheduled multi-platform campaign out

YOU WILL MASTER

Adapter architecture ·
idempotency · scheduling ·
review workflow

YOUR \$0 STACK

Node or Python · Telegram or
Discord or Mastodon · SQLite or
Docker Postgres

How to read this document: Sections 1–2 tell you what this capstone is. Sections 3–5 tell you what to build and when the work is complete. Sections 6–9 give the build phases, the free tools, the GitHub rules, and how to submit. Section 10 tells you how your work is checked. Sections 11–12 give resources and the glossary. This document is standalone. You do not need other assignments, and you do not need files from us.

Contents

1. The project	1	7. Free tools	5
2. Rules	2	8. GitHub rules	6
3. What you build	2	9. How to submit.....	6
4. Architecture.....	3	10. How your work is checked	7
5. Requirements	4	11. Resources	8
6. Build in phases	4	12. Glossary	8

1 · The project

You build a service that changes one blog post into a social media campaign. The service makes one **variant** for each platform. Each platform has different rules for length, tone, and hashtags. A person approves each variant. A scheduler publishes each approved variant at a set time, through one clean **adapter** interface.

The point of this capstone is not the API calls. The point is a publishing system that survives the real world. A retry must not publish the same post two times. A worker that stops in the middle of a batch must continue safely. An unapproved variant must never go out. Marketing teams pay for this exact system, because no tool does it well.

The hard parts

- **Idempotency.** A retry after a timeout must not make a second post. This is the heart of the grade.
- **Constraint profiles.** Each platform has rules. Your code must enforce the rules. Hope is not enforcement.

- **Durable scheduling.** A worker that stops mid-batch must continue without duplicate posts.
- **The adapter seam.** Your app must not know which platform it publishes to. A new platform is a new adapter, not a rewrite.

2 · Rules

Rule	What it means for you
Standalone build	This capstone is complete in itself. You do not need work from other assignments. You do not need files, servers, or code from us. Every tool in this document is public and free.
One public repo	The capstone lives in its own public GitHub repository from day one. See Section 8.
\$0, no credit card — ever	Everything can be built with free tools. Section 7 lists the exact free stack. If a page asks you for a credit card, stop. You took a wrong turn. The free path exists.
Real accounts, safe targets	Publish only to targets that you own and that permit bots: your own Telegram channel, your own Discord server, or a Mastodon test account. Do not publish to Instagram, X, or LinkedIn. For those platforms, you write mock adapters.
Use AI, own it	AI tools are allowed. Keep <code>BUILDLOG.md</code> honest: where AI helped, and where AI was wrong. You must be able to explain each line of your code.
Secrets stay secret	Bot tokens go in <code>.env</code> , never in a commit. Ship a <code>.env.example</code> with placeholder values.

3 · What you build

Give the system one published blog post. The post can be a URL or pasted text. The system then does five things.

1 Post ingestion

Accept a blog post as a URL or as pasted Markdown. Store the post. The stored post is the single **source of truth**. All generation reads from the stored post only.

2 Variant generation with constraint profiles

Make one variant for each target platform. Examples: a short post for X style, a longer post for LinkedIn style, a caption for Instagram style. Each platform has a **constraint profile**: maximum length, tone, and hashtag count. Your validation code enforces each profile. A variant that breaks a rule never reaches review.

You can generate the text with a free AI model (Gemini free tier, or local Ollama). You can also use hand-written templates. The AI is optional. The enforcement is graded.

3 Review workflow

Each variant has a status: `draft`, `approved`, `rejected`, or `published`. A person approves, edits, or rejects each variant. Only an approved variant can be scheduled. A schedule request for an unapproved variant gets a 4xx status code with an error message.

4 Publisher adapter layer

One `SocialPublisher` interface. At least three implementations:

```
SocialPublisher
+-- TelegramPublisher    (or Discord / Mastodon – one REAL free target)
+-- MockXPublisher       (writes what it WOULD post to a local store)
+-- MockLinkedInPublisher
```

You write the mock adapters yourself. A mock adapter records the post in your database and shows a preview. The application depends on the interface only. A swap from one adapter to another touches zero business logic.

5 Idempotent scheduling and publish history

A calendar of time slots. A scheduler publishes each approved variant at its time, through the adapter. The publish is **idempotent**: the same variant in the same slot posts one time, even under retries. A worker crash mid-batch does not cause a duplicate post. Each publish attempt lands in a visible **publish history**.

Not in scope: image generation, analytics, engagement tracking, and real Instagram, X, or LinkedIn accounts. The mock adapters plus one real free platform prove the architecture.

4 · Architecture

Content flows down the left side. The reliability machinery guards the right side. Every publish goes through the same interface.

```
[blog post: URL or markdown]
  |
  v
ingest + store ---> variant generator ---> constraint validation
                        |                      |
                        v                      v
                    review workflow: draft -> approved | rejected
                        |
                        v
                    scheduler (durable, resumable)
                        |
                        v
                    SocialPublisher interface
                    +-- Telegram / Discord / Mastodon (real)
                    +-- MockX + MockLinkedIn           (yours)
                        |
                        v
                    publish history: one slot = one post, always
```

5 · Requirements

Done = every box ticked. Section 10 shows how we check each box.

- ☐ **Ingestion:** a post enters as a URL or as Markdown, and is stored. Generation reads only the stored post.
- ☐ **Constraint profiles enforced by code:** length, tone rules, and hashtag count per platform. Prove it in `EVIDENCE.md`: a test output or a command transcript that shows a bad variant is blocked.
- ☐ **Review workflow:** statuses `draft / approved / rejected / published`. Only approved variants can be scheduled. An unapproved schedule attempt returns a 4xx status code with an error message.
- ☐ **Adapter layer:** one `SocialPublisher` interface, one real free platform, and at least two mock adapters. An adapter swap changes configuration, not business logic. Prove it in `EVIDENCE.md`.
- ☐ **Idempotent publish:** the same variant and slot never post two times, even under retries. Prove it in `EVIDENCE.md`: a test output or a repeated-call transcript.
- ☐ **Durable scheduling:** a worker restart mid-batch continues with zero duplicate posts.
- ☐ **Publish history:** each attempt is recorded and visible, with its result.
- ☐ **Secrets clean:** tokens live in `.env` only. The repo has a `.env.example`.
- ☐ **README:** what the system does, an architecture diagram, and exact run steps. A stranger can run it with one command.

6 · Build in phases

This capstone is self-paced. There is no calendar and no deadline. Work through the phases in order. Each phase ends with a gate: a concrete result that tells you it is safe to move on. Short on time? Shrink the number of platforms, not the phases.

1 Design

approx. 4–6 h

One page of thinking saves a week of rework.

- Write the constraint profile for each platform: length, tone, hashtags.
- Design the `SocialPublisher` interface signature.
- Design the data model: posts, variants, statuses, slots, publish attempts.

GATE — a one-page design document is in the repo: problem, data model, API surface, and one explicit non-goal.

2 Ingestion and generation

approx. 8–12 h

One post in, honest variants out.

- Build the ingest endpoint: URL fetch or pasted Markdown, then storage.
- Build the variant generator for two platforms. Templates are fine. AI is optional.
- Build the validation that enforces each constraint profile.

GATE — one stored post produces two different variants, and a rule-breaking variant is blocked with an error message that names the broken rule.

3 Review workflow

approx. 6–8 h

Nothing publishes unapproved. Ever.

- Add the variant statuses and the approve, edit, and reject endpoints.
- Refuse a schedule request for an unapproved variant with a 4xx status code and an error message.

GATE — an unapproved variant cannot be scheduled. An approved one can.

4 Adapters and idempotent publish

approx. 10–14 h

The heart of the grade lives here.

- Define the `SocialPublisher` interface.
- Build one real adapter: Telegram bot, Discord webhook, or Mastodon API.
- Build two mock adapters that record what they would post.
- Make the publish idempotent with an idempotency key per variant and slot.

GATE — a real message lands in your own channel. A repeated publish call creates exactly one post.

5 Scheduling, history, and hardening

approx. 8–12 h

Now make it survive the real world.

- Build the scheduler: a job store plus a worker that publishes due variants.
- Stop the worker mid-batch, restart it, and prove zero duplicates.
- Build the publish history view and finish the README.

GATE — every Requirements box is ticked.

FINAL SELF-CHECK — go through the Requirements list in Section 5. Tick every box. Check your proofs in `EVIDENCE.md`.

★ Stretch goals

optional · only if the core ships

- ☐ A/B variants per platform, with a pick-the-winner flow.
- ☐ A grounding check: each claim in a variant must exist in the source post. A planted fake statistic gets caught.
- ☐ Cost per campaign, tracked and shown, if you use an AI model.
- ☐ Multi-tenant: agencies with isolated clients.
- ☐ A test suite for the scary cases: blocked variant, refused schedule, duplicate publish, adapter swap.

7 · Free tools

Everything you need, free and without a credit card. Every item is public. Nothing comes from us.

You need	Free tool (no credit card)	Notes
----------	----------------------------	-------

Language + framework	Node.js + Express, or Python + FastAPI	Free, either lane
Real publish target	Telegram Bot API · Discord webhook · Mastodon	Free · you own the channel · bots are permitted
Mock platforms	Your own mock adapters	They record posts in your database and render a preview
Database	SQLite, or PostgreSQL via Docker	Free
Variant text (optional AI)	Gemini free tier, or local Ollama, or hand-written templates	Free · AI optional · enforcement is what is graded
Scheduling / queue	BullMQ + Redis (Docker), or APScheduler with a persistent job store	Free
Secrets	<code>.env</code> file + <code>.env.example</code>	Built-in / free

The iron rule: if a tool, a tier, or a tutorial asks for a credit card, it is the wrong path. A free alternative for this capstone exists in the table above. Stuck? Ask in the community before you pay for anything.

8 · GitHub rules

Your submission is your public GitHub repository. That is the full submission. Reviewers examine the repository as it stands.

- **One dedicated repository, public from day one.** Name it `flyrank-capstone-social-studio`. Do not build in a private repo that you flip later. Progress in the open is part of the story.
- **Commit as you build.** Small commits, clear messages. Each phase of Section 6 should be visible in the history.
- **Never commit a secret.** Put `.env` in `.gitignore` before the first commit. Ship `.env.example` instead. If a key leaks, rotate the key and ask for help.
- **A stranger can run it.** The README gives one run command (for example `docker compose up`) plus a seed step for sample data.

Required files at submission

File	Contents
<code>README.md</code>	What the system does, an architecture diagram, exact run and seed steps, and known limitations.
<code>EVIDENCE.md</code>	One proof for each Requirements box: a test name with its output, a log line, or a curl transcript.
<code>BUILDLOG.md</code>	Your AI-usage log: where AI helped, where it was wrong, what you changed. Honesty is graded, perfection is not.
<code>.env.example</code>	Every environment variable, with safe placeholder values.

9 · How to submit

- Submissions go through the portal submission form.
- The intern must create a new public GitHub repository with their code.
- They paste the link to the repository into the submission form on the portal.
- Do not upload ZIP files, ZIP folders, or the full codebase into the form. This is the most common cause of submission errors.

Review is asynchronous. Nothing is scheduled with you. If we need anything from you, we will reach out through the portal.

10 · How your work is checked

The checks are published up front. You know exactly what will be checked, so build to pass it.

Acceptance probes (pass / fail)

A reviewer runs these against your system, with your README as the only guide:

PROBE 1 — Ingest a sample post. The system generates variants for each configured platform, and each variant passes its constraint profile.

PROBE 2 — Create a variant that breaks a platform rule. The validation blocks it with an error message that names the broken rule, before review.

PROBE 3 — Try to schedule an unapproved variant. The system refuses with a 4xx status code and an error message.

PROBE 4 — Approve a variant and schedule it two minutes out. The scheduler publishes it to the real free target. The publish record links to the live message.

PROBE 5 — Force a publish retry: stop the worker mid-publish, then restart. The history shows exactly one successful post. No duplicates.

PROBE 6 — Swap the adapter in configuration, for example `telegram` to `mock_x`. The same campaign publishes through the mock. No code change outside the adapters.

One principle guides the review: a small system that is correct, resilient, and well tested beats a big one that falls over.

11 · Resources

Do not read everything. Each row says when you need it. All resources are free, with no credit card.

Resource	Format	When you need it
Telegram Bot API docs	Reference	Phase 4, if Telegram is your real target. <code>sendMessage</code> is all you need.
Discord webhook docs	Reference	Phase 4, if Discord is your real target. One POST request publishes a message.
Mastodon API docs	Reference	Phase 4, if you want real OAuth practice with your target.
Adapter pattern — Refactoring Guru	Article, ~15 min	Before Phase 4. The pattern that carries this capstone.
Idempotency — MDN glossary	Article, ~5 min	Before Phase 4. What "safe to retry" means, precisely.
BullMQ docs	Reference	Phase 5, JavaScript lane: repeatable jobs with Redis.
APScheduler docs	Reference	Phase 5, Python lane: use a persistent job store, not the default memory store.
Gemini API free tier	Reference	Phase 2, only if you generate variants with AI. The free tier needs no card.
Ollama	Tool	Phase 2, if you prefer a local model over an API.
The Twelve-Factor App — Config	Article, ~10 min	Why secrets live in the environment, not in code.

12 · Glossary

Plain definitions of the **bold** words above. No definition depends on another.

Word	What it means
Variant	One platform-specific version of the blog post. Example: the short X-style version of one article.
Constraint profile	The set of rules for one platform: maximum length, tone, and hashtag count. Your code enforces the rules.
Adapter	A class that translates between your app's one interface and one platform's specific API. Your app talks to the interface only.
Mock adapter	An adapter that does not call a real platform. It records what it would post in your database, and shows a preview.
Idempotency	The property that a repeated action has the effect of one action. A retried publish makes one post, not two.
Idempotency key	A unique value per variant and slot. The publisher uses it to detect and ignore a repeated attempt.
Durable scheduling	Scheduled work that survives a crash. The job store lives in a database, so a restarted worker continues correctly.
Source of truth	The one stored copy of the post that all generation reads. Nothing else feeds the generator.

Publish history	The record of each publish attempt and its result. One approved variant and slot shows exactly one success.
Review workflow	The status path of a variant: draft, then approved or rejected, then published. Only approved variants can be scheduled.

FlyRank Internship · Backend Development Track · Capstone — Social Media Studio. All linked resources are free, with no credit card required. This brief is standalone: you need only this document and public resources.